

附上部分与 codex 的对话

我要完成一份最基础的opencv颜色分割的实验 我...

我现在有一个视频内容，是一个在灰白色桌面上的亮橙色橘子，我现在需要用上述所有知识，选取橘子作为框选目标，能够合理运用颜色分割、最小框选和多边形拟合将目标框选出来，且能够实时对视频进行处理，并将框选结果可视化出来（imshow视频出来） 视频大概五秒钟，视频名字为test.mp4

第一部分：需求确认

我理解这次实验目标是：

1. 输入视频文件名为 test.mp4 。
2. 视频内容是灰白色桌面上的亮橙色橘子。
3. 框选目标是**橘子本体**，不是桌面、阴影、高光或其它背景内容。
4. 需要综合使用：
 - OpenCV 读取视频；
 - HSV 颜色分割提取橙色区域；
 - findContours 查找轮廓；
 - approxPolyDP 做多边形拟合；
 - contourArea 筛选面积；
 - boundingRect 或类似方式做最小框选；
 - imshow 实时显示处理后的视频画面。
5. 暂时只要求实时显示框选结果，不要求保存新视频文件。
6. 程序运行时可以按键退出，例如按 q 退出，视频播放结束后自动关闭窗口。

如果以上理解无误，请回复：**该部分已确认**

我收到后进入第二部分，先分析视频版和图片版代码相比需要补哪些关键步骤。



好的 并且视频内容大概如截图所示 我确认

要求后续变更

+ 完全访问权限 5.5 超高

效果不错 但是橘子的轮廓的边数略少 能再多点吗

可以。我的理解是：你现在只想让绿色的多边形拟合轮廓更贴近橘子圆形，也就是拟合点/边数更多一些，但其它部分保持不变：

1. 橙色颜色分割不改；
2. 红色最小框选不改；
3. 视频实时显示不改；
4. 只调整 `approxPolyDP` 的拟合精度，让轮廓边数更多。

原因一般是这一行里的 `0.02` 偏大：

```
python
epsilon = 0.02 * cv2.arcLength(max_contour, True)
```

如果你确认只优化这一处，请回复：该部分已确认
我再给你具体替换代码。

确认

确认

把这一行：

```
python
epsilon = 0.02 * cv2.arcLength(max_contour, True)
```

改小即可，比如：

```
python
epsilon = 0.005 * cv2.arcLength(max_contour, True)
```

完整局部如下：

```
python
if area > 1000:
    epsilon = 0.005 * cv2.arcLength(max_contour, True)
    approx = cv2.approxPolyDP(max_contour, epsilon, True)

    x, y, w, h = cv2.boundingRect(approx)

    cv2.rectangle(frame, (x, y), (x + w, y + h), (0, 0, 255), 2)
    cv2.drawContours(frame, [approx], -1, (0, 255, 0), 2)
```

`epsilon` 越小，拟合轮廓越接近原始轮廓，边数也会更多。这个改动只影响绿色轮廓，不会改变颜色分割和视频读取流程。

📄 👍 🔍 ↺ 0:13

已处理 16s >

第一部分：需求确认

我理解这次任务是：

1. 输入视频： `test3.mp4`
2. 视频内容：白纸上有一个矩形，白纸持续变换方位。
3. 检测目标：白纸上的矩形四个角点。
4. 使用方法：结合颜色/二值化、轮廓检测、最小框选、多边形拟合，得到矩形四角点。
5. 位姿解算：用四个角点作为 `imagePoints`，用矩形在物体坐标系下的四个点作为 `objectPoints`，调用 `cv2.solvePnP`。
6. 可视化结果：把三维坐标轴投影回视频画面，显示矩形的三维位姿。
7. 输出视频：保存为 `mp4_out3.mp4`。
8. 最终需要一份可运行代码，并生成处理后的视频文件。

这里有两个关键点需要你确认：

1. 你是否有这个矩形的真实长宽？如果没有，我会用“假定矩形宽 100、高 60”的相对尺寸做姿态演示，位姿方向可视化是合理的，但平移距离不是严格真实物理距离。
2. 你是否有相机标定得到的 `cameraMatrix` 和 `distCoeffs`？如果没有，我会根据视频分辨率构造一个近似相机内参，并把畸变系数设为 0，用于基础实验演示。

如果你同意以上理解和这两个默认处理方式，请回复：该部分已确认。

📄 🍷 🔍 ↻ 15:44



矩形长宽为3cm×4cm，视频为4k60fps 视频截图如图所示 其余内容确认

已处理 0s >

第二部分：参数确认

你补充的信息我已收到：



我要完成一份最基础的opencv颜色分割的实验 我...



如果这一部分确认无误，请回复：**该部分已确认**
我再进入第三部分，说明检测矩形四角点的算法流程。

确认

15:47

第三部分：算法流程确认

针对你这个视频，矩形是白纸上的黑色线框，所以检测流程会这样做：

1. 逐帧读取 `test3.mp4`。
2. 转灰度图。
3. 用高斯模糊降低噪声。
4. 用阈值或边缘检测提取黑色矩形线条。
5. 用形态学闭运算连接断裂边缘。
6. 使用 `findContours` 查找轮廓。
7. 用 `contourArea` 过滤过小噪声和过大背景。
8. 用 `approxPolyDP` 做多边形拟合。
9. 只保留拟合后有 4 个角点的轮廓。
10. 按左上、右上、右下、左下的顺序排序四个角点。
11. 将四个角点作为 `imagePoints` 输入 `solvePnP`。
12. 用 `projectPoints` 投影三维坐标轴。
13. 在画面中画出：
 - 绿色矩形轮廓；
 - 红色最小框选；
 - 三维坐标轴；
 - 简单的 `tvec` 信息。
14. 写入输出视频 `mp4_out3.mp4`。

考虑到原视频是 4K 60fps，为了代码运行更流畅，我建议：

- 处理时可以按原始分辨率读取；
- 但 `imshow` 展示时缩小到合适大小；



```
show_frame = cv2.resize(frame, (960, 540))
cv2.imshow("solvePnP result", show_frame)

if cv2.waitKey(1) & 0xFF == ord("q"):
    break

cap.release()
out.release()
cv2.destroyAllWindows()
```

这里红框是最小框选，绿色线是检测到的矩形轮廓，红/绿/蓝三条轴分别表示投影回图像中的三维坐标轴。输出视频会保存为 `mp4_out3.mp4`。

📄 👍 🔍 ↻ 15:48

视频为竖屏4k60fps 更改代码

第五部分：竖屏视频修改确认

竖屏 4K 60fps 主要影响两处：

1. `imshow` 展示尺寸不能再固定成 `(960, 540)`，否则竖屏会被压扁。
2. 近似相机内参里焦距建议用 `max(frame_width, frame_height)`，竖屏时也更合理。

也就是说，其它检测流程、`solvePnP`、输出文件名 `mp4_out3.mp4` 都不需要改。

请确认这次只修改这两处：

```
python

focal_length = max(frame_width, frame_height)
```

以及竖屏自适应显示尺寸。

如果确认，请回复：该部分已确认

我再给你修改后的代码。

我要完成一份最基础的opencv颜色分割的实验 我...

```
cameraMatrix = np.array([
    [frame_width, 0, frame_width / 2],
    [0, frame_width, frame_height / 2],
    [0, 0, 1]
], dtype=np.float32)
```

改成:

```
python

focal_length = max(frame_width, frame_height)

cameraMatrix = np.array([
    [focal_length, 0, frame_width / 2],
    [0, focal_length, frame_height / 2],
    [0, 0, 1]
], dtype=np.float32)
```

第二处, 把原来的显示缩放:

```
python

show_frame = cv2.resize(frame, (960, 540))
cv2.imshow("solvePnP result", show_frame)
```

改成:

```
python

show_height = 960
show_width = int(frame_width * show_height / frame_height)

show_frame = cv2.resize(frame, (show_width, show_height))
cv2.imshow("solvePnP result", show_frame)
```

这样竖屏 4K 视频显示时不会被横向压扁, solvePnP 使用的近似内参也更适合竖屏画面。

📄 🌟 🔍 ↻ 15:51

我现在需要为我的项目撰写README.md 项目是桌面上的bjtu文件夹里的25-26-2文件夹里的rmtest文件夹 使用的python版本是3.11 额外添加了opencv-python 和 opencv-contrib-python两个解释器 我的readme.md文件已经存在于rmtest文件夹中了，你来读取这个文件夹中的所有文件并写出一份标准的readme文件，允许你读取这个文件夹中的所有文件但只允许你更改README.md

17:00

已处理 7m 13s >

第一部分：README 需求确认

我已读取 `rmtest` 文件夹内容，确认当前项目包含：

- `test0.py` : OpenCV 读取并显示图片基础实验
- `test1.py` : HSV 颜色分割，输出 `test1_out.png`
- `test2.py` : 箭头状灯条颜色分割、轮廓检测、多边形拟合、最小框选，输出 `test2_out.png`
- `additional_test.py` : 橘子视频颜色分割、轮廓拟合、实时框选
- `test3.py` : 矩形检测、`solvePnP` 位姿解算、坐标轴可视化，输出 `test3_out.mp4`
- 输入素材: `test0.jpg`、`test1.png`、`test2.png`、`additional_test.mp4`、`test3.mp4`
- 当前 `README.md` 为空
- Python 版本: `3.11`
- 依赖: `opencv-python`、`opencv-contrib-python`，代码中还实际使用了 `numpy`

我准备只修改：

`C:\Users\Jiang Haoru\Desktop\bjtu\25-26-2\rmtest\README.md`

README 会按标准项目文档写成这些板块：

1. 项目简介
2. 环境要求
3. 安装依赖
4. 项目结构



5. 实验说明

要求后续变更

我把additional_test.mp4文件替换成了竖屏4k60fps的文件 但是additional_test.py这个脚本执行出来后出现的窗口大小明显不对 帮我纠正additional_test.py文件中的错误

17:14

已处理 33s >

第一部分：问题确认

我已读取文件和视频信息，当前新视频是竖屏 4K：

```
text

width = 2160
height = 3840
fps ≈ 60
frames = 303
```

问题出在 additional_test.py 里直接执行了：

```
python

cv2.imshow("orange tracking", frame)
```

也就是把 2160×3840 的原始竖屏帧直接显示出来，所以窗口会明显过大、不适合屏幕显示。

我准备只修改 additional_test.py，并且只改显示相关逻辑：

1. 保留原始帧用于颜色分割、轮廓检测和框选；
2. 在 imshow 前额外生成一个按比例缩小的 show_frame；
3. 竖屏显示高度设为合适值，例如 960；
4. 保持视频比例不变，避免画面被压扁；
5. 不修改 additional_test.mp4，不修改其它文件。

如果确认，请回复：该部分已确认

我再进入下一部分，实际修改 additional_test.py。



